#9 If the recursive call returned `null`, we only need to update this node's bounding envelope, and then we can in turn return `null` as well.
#10 Otherwise, it means that `closestchild` has been split: we need to remove it from the list of children...
#11/#12 ... and add the two newly generated spheres in its place.
#13 We need to compute the centroid and radius for this node.
#14 If the number of children is still within the max allowed, we are done with backtracking.
#15 If it gets here, it means that the node needs to be split: create two new nodes and return them.

Therefore, we will give an explicit implementation of the tree's method for `insert`; remember, we will actually only expose methods defined on the data structure classes (`KdTree`, `SsTree` etc...) and not on the nodes' classes (such as `SsNode`), but we usually omit the formers when they are just wrappers around the nodes' methods). Take a look at listing 10.7 to check out how we can handle root splits – also, let me highlight this again: this code snippet is the only point where our tree's height grows.

Listing 10.7 The SsTree::insert method

```
function
SsTree::insert(point)
      #1
  (newChild1, newChild2) ← insert(this.root,
point)                    #2
  if newChild1 != null then
                                                    #3
```